TECHNOVA SOLUTIONS PVT. LTD.

GIT WORKFLOW

Document ID	TN-ENG-STD-003
Document Type	Engineering Git Workflow
Document Owner	Engineering
Approving Authority	VP Engineering / CTO
Classification	Internal — Engineering
Version	1.0
Effective Date	01 April 2026
Review Cycle	Annual
Status	Active
Supersedes	New Standard

1. Purpose

This Git Workflow defines the standard process for managing source code changes at TechNova Solutions Pvt. Ltd. (“TechNova” or “the Company”). It establishes expectations for repositories, branches, commits, pull requests, reviews, merging, releases, tags, hotfixes, and repository maintenance.

The workflow is intended to make changes traceable, reviewable, reproducible, and safe while allowing engineering teams to deliver software efficiently.

2. Scope

This workflow applies to Engineering repositories containing application code, services, libraries, infrastructure code, automation, data-processing code, configuration, and other maintained technical artifacts.

Teams may use a documented variation where a technology or delivery model requires it, provided that security, traceability, review, and access-control requirements remain satisfied.

3. Git Workflow Principles

- Every meaningful production change should be traceable to a source-control change.
- Changes should be small enough to review effectively where practical.
- The main production branch should remain stable and releasable.
- Code should be reviewed before integration according to project requirements.
- Branches should have clear ownership and purpose.
- Repository history should remain understandable.
- Credentials and sensitive information must never be committed.
- Release and deployment activities should be reproducible.

4. Repository Ownership

Every important repository should have an identified owning team or technical owner.

- Maintain repository documentation.

- Define contribution and review expectations.
- Maintain appropriate branch protection.
- Manage repository permissions.
- Review stale branches and access.
- Coordinate repository migrations or archival when required.

5. Repository Structure

Repositories should use a predictable structure appropriate to the technology.

- Keep source code organized by logical responsibility.
- Separate tests from production code where the project convention requires it.
- Maintain configuration and documentation in appropriate locations.
- Do not commit unnecessary local-development artifacts.
- Provide a clear README or equivalent project documentation.

6. Repository Access

Repository access must follow the Access Control Policy and the principle of least privilege.

- Grant access only to users who require it.
- Use individual accounts.
- Restrict administrative permissions.
- Review repository access when team membership changes.
- Remove access when the business need ends.

7. Main Branch

The main branch represents the primary integrated codebase and should normally remain in a buildable and releasable condition.

- Direct pushes to protected main branches should be restricted.
- Changes should normally enter through the approved review workflow.
- Automated checks should run before integration where configured.
- Broken builds should be addressed promptly.

8. Branch Types

TechNova teams may use the following logical branch categories according to project needs:

Branch Type	Purpose
Main	Stable integrated code and the source for releases where applicable.
Feature	Development of a specific feature or bounded change.
Bugfix	Correction of a defect that does not require an emergency production process.
Release	Preparation and stabilization of a planned release where the project uses release branches.
Hotfix	Urgent production correction requiring an expedited workflow.

9. Feature Branches

Feature branches should represent a specific piece of work or a coherent change.

- Create the branch from the appropriate current base branch.
- Use a descriptive branch name.
- Keep the branch focused on its intended work.
- Synchronize with the base branch as appropriate to reduce integration surprises.
- Delete the branch after successful integration when it is no longer required.

10. Branch Naming

Branch names should be descriptive, consistent, and easy to understand.

- Use the team's approved naming convention.
- Include a work-item identifier when the project uses one.
- Use concise descriptive wording.
- Avoid spaces and ambiguous names.

Example conventions may include names such as feature/TN-142-user-search, bugfix/TN-207-timeout-handling, or hotfix/TN-311-authentication-failure. Teams should use the convention formally adopted by their repository.

11. Starting a Change

- Understand the requirement or issue before coding.
- Confirm the correct base branch.
- Create a dedicated branch when the workflow requires one.
- Update the branch from the base when necessary.
- Make the change according to Coding Standards.

12. Commit Standards

Commits should communicate meaningful units of work.

- Use a concise and descriptive subject.
- Explain important reasoning in the commit body when useful.
- Keep unrelated changes out of the commit.
- Do not commit temporary debugging artifacts.
- Do not commit secrets or sensitive information.

13. Commit Frequency

Commit frequency should support reviewability and recoverability. Engineers should avoid both extremely large unstructured commits and excessive commits that provide no meaningful history.

Before a pull request is finalized, commits may be reorganized or squashed when the repository workflow permits it.

14. Commit Messages

A recommended commit message structure is a short imperative subject followed by an optional explanatory body.

- Use consistent terminology.
- Reference a work item when required.
- Explain why a non-obvious change was made.

- Avoid messages such as “stuff”, “changes”, or “final” that provide no useful context.

15. Pull Requests

A pull request is the primary mechanism for proposing a change for review where the repository uses pull requests.

- Identify the purpose of the change.
- Describe important implementation details.
- Identify testing performed.
- Mention known limitations, risks, or follow-up work.
- Link the relevant work item where applicable.

16. Pull Request Scope

Pull requests should normally contain one coherent change or a logically connected set of changes.

Large changes should be divided into smaller reviewable units when technically and operationally practical.

17. Pull Request Description

- What problem does this change solve?
- What was changed?
- How was it tested?
- Are there deployment or migration considerations?
- Are there security, performance, or compatibility implications?

18. Code Review

Pull requests must receive the level of review required by the repository or project.

- Review correctness and intended behavior.
- Review maintainability and clarity.
- Consider security and data handling.
- Consider test coverage and regression risk.
- Check compatibility and operational impact where relevant.

Detailed review expectations are defined in the separate Code Review Guidelines document.

19. Review Comments

- Review comments should be specific and constructive.
- Authors should respond to material comments.
- Disagreements should be resolved through technical reasoning and appropriate escalation.
- Material unresolved concerns should not be silently ignored.

20. Automated Checks

Repositories should use appropriate automated checks before merging.

- Formatting
- Linting

- Unit tests
- Integration tests where applicable
- Static analysis
- Dependency or security checks where configured
- Build verification

21. Merge Requirements

A change should be merged only after required reviews and automated checks have been satisfied or an authorized exception has been recorded.

- Required approvals are present.
- Required checks are successful.
- Material conflicts have been resolved.
- The pull request accurately represents the final change.
- Any required documentation has been updated.

22. Merge Strategies

Repositories may use merge commits, squash merging, rebase-based workflows, or another approved strategy.

The repository owner should document the chosen strategy and apply it consistently enough to preserve understandable history.

23. Resolving Merge Conflicts

- Understand both sides of the conflicting change before resolving.
- Do not resolve conflicts mechanically without checking behavior.
- Run appropriate tests after conflict resolution.
- Ask the original author or owner when intent is unclear.

24. Keeping Branches Current

Long-lived branches should be synchronized with their base branch regularly where practical.

Frequent synchronization reduces the size of eventual merge conflicts and helps expose integration problems earlier.

25. Rebase and History Rewriting

History rewriting may be used on private feature branches when it improves clarity and the workflow permits it.

- Do not rewrite shared history without authorization.
- Do not force-push protected branches.
- Use force-push only where the repository policy permits it.
- Communicate when rewriting a branch that other engineers may be using.

26. Protected Branches

Important branches should use repository protection controls where supported.

- Restrict direct pushes.
- Require pull-request review.

- Require automated checks.
- Restrict force-push and branch deletion.
- Limit administrative overrides.

27. Release Branches

Projects using release branches should create them from a stable point and limit changes to release preparation, approved fixes, and other explicitly permitted work.

Release branches should not become an uncontrolled parallel development path.

28. Release Tags

Production releases should be identifiable through version tags or another approved immutable release identifier.

- Use a consistent versioning convention.
- Tag the exact source used for the release.
- Do not move or reuse release tags without an approved exceptional process.
- Associate release identifiers with deployment records where applicable.

29. Versioning

Projects should use an appropriate versioning scheme for releases and artifacts.

Where semantic versioning is adopted, teams should follow the project's documented interpretation of major, minor, and patch changes.

30. Hotfix Workflow

A hotfix is an urgent correction for a significant production problem.

- Create the hotfix from the appropriate production source.
- Keep the change narrowly focused.
- Perform the highest-priority feasible validation.
- Obtain the required expedited review or approval.
- Deploy through the approved emergency or hotfix process.
- Merge the fix back into relevant development branches where necessary.

31. Emergency Changes

Emergency changes may use an expedited workflow when delaying the change would create unacceptable operational or security risk.

Emergency changes must still be traceable and should receive retrospective review and documentation where required.

32. Pull Request Approval and Ownership

Approvers should have sufficient technical or domain understanding to evaluate the change.

Authors should not approve their own changes where the repository requires independent review.

33. Draft Pull Requests

Draft pull requests may be used to request early feedback or communicate work in progress.

Draft changes should not be merged until normal merge requirements are satisfied.

34. Review Before Merge

- Confirm the intended files changed.
- Check for accidental debug code.
- Check for secrets or sensitive data.
- Verify tests and checks.
- Review migration or configuration changes.
- Confirm documentation is updated when necessary.

35. After Merge

After integration, engineers should monitor relevant build, test, deployment, and operational signals.

If a merged change causes a significant problem, the team should follow the incident, rollback, or corrective-change process.

36. Reverting Changes

When a merged change causes unacceptable behavior, the team may revert it using an appropriate version-control mechanism.

- Prefer a traceable revert rather than deleting history.
- Document the reason for the revert when material.
- Create a follow-up correction where appropriate.
- Ensure the source of truth remains understandable.

37. Rollback and Git

Git history should support identifying the source of a deployed version and, where operationally appropriate, returning to a known-good version.

Deployment and rollback mechanisms are defined in the separate Deployment Policy.

38. Repository Tags and Artifacts

Release tags, package versions, and build artifacts should identify the source revision used to produce them where feasible.

39. Generated Files

Generated files should be committed only when required by the repository's build or distribution model.

Generated artifacts that can be reliably reproduced should generally be generated through the build process rather than manually edited.

40. Sensitive Information

- Never commit passwords, API keys, access tokens, private keys, certificates, or other secrets.
- Do not commit production data unless explicitly authorized and appropriately protected.
- Do not commit confidential documents or personal data unnecessarily.
- If sensitive information is committed accidentally, treat it as a potential security incident and follow Information Security procedures.

41. Dependency and Lock Files

Dependency lock files should be maintained according to the package manager and repository requirements.

Changes to dependencies should be reviewed for security, compatibility, licensing, and reproducibility.

42. Submodules and External Repositories

Git submodules or external repository dependencies should be used only when they provide a clear benefit and are documented.

Referenced revisions should be controlled so that builds remain reproducible.

43. Monorepositories

In monorepositories, teams should define ownership and review boundaries for different components.

Changes affecting shared libraries or multiple services should receive the appropriate cross-team review.

44. Repository Maintenance

- Archive obsolete repositories according to Company procedures.
- Remove stale branches when safe.
- Review repository permissions periodically.
- Keep repository documentation current.
- Maintain required CI/CD configuration.

45. Forks and Copies

Creating copies or forks of Company repositories must follow repository permissions, security requirements, and intellectual-property rules.

Private repositories and sensitive code must not be copied to unauthorized locations.

46. Git Hooks and Local Automation

Local hooks and automation may improve developer experience, but important quality or security controls should also be enforced centrally where practical.

A developer must not bypass centrally enforced security or quality controls through local configuration.

47. CI/CD Integration

Git repositories may integrate with automated build, test, security scanning, artifact, and deployment systems.

- Protect CI/CD credentials.
- Restrict deployment permissions.
- Make pipeline changes reviewable.
- Keep production deployment controls separate from ordinary source-code permissions where appropriate.

48. Access to Production Branches

Access to branches containing production-ready or release code should be restricted according to role and responsibility.

Privileged repository actions should remain attributable to individual users.

49. Auditability

Repository activity should provide sufficient history to understand who made, reviewed, merged, or released important changes.

Audit records should be protected according to applicable security and retention requirements.

50. Frequently Asked Questions

Can I push directly to main?

Only where the repository explicitly permits it. Protected production branches should normally require the approved review and automated-check workflow.

How large should a pull request be?

Small enough to review effectively where practical. Large work should be divided into coherent changes when technically possible.

Can I rewrite my feature branch history?

Usually, if the branch is private and the repository workflow permits it. Do not rewrite shared or protected history without authorization.

What should I do if I accidentally commit a secret?

Treat it as a potential security incident, report it promptly, and follow the Information Security response process. Removing the file from the latest commit alone may not be sufficient.

Can a hotfix skip all review?

Emergency changes may use expedited review, but they should remain traceable and follow the approved emergency process.

When should I delete a feature branch?

After successful integration when it is no longer needed, unless the repository has a reason to retain it.

What should a pull request description contain?

The purpose, key changes, testing performed, and important risks, deployment considerations, or follow-up work.

Who should review my code?

Approvers should have sufficient technical or domain knowledge and follow the repository's review requirements.

What happens if a merge breaks production?

Follow the incident and rollback/correction procedures, then investigate and address the underlying issue.

Where are detailed code-review rules defined?

In the separate Code Review Guidelines document.

51. Workflow Administration

Engineering owns this Git Workflow and reviews it at least annually or when material changes occur in source-control platforms, development practices, security requirements, or delivery processes.

The latest approved version maintained in the official Company repository is the authoritative version.

52. Revision History

Version	Effective Date	Description	Owner	Approval
1.0	01 April 2026	Initial release of Engineering Git Workflow	Engineering	VP Engineering / CTO

53. Related Documents

- Engineering Handbook
- Coding Standards
- Code Review Guidelines
- Deployment Policy
- Testing Standards
- Information Security Policy
- Access Control Policy

END OF DOCUMENT

TN-ENG-STD-003 | Version 1.0 | Internal — Engineering | Engineering